

PRATICAL-08

Aim: Write a program to analyze the web server log stream data using Apache Flume Framework.

Introduction: Apache Flume is a distributed, reliable, and available system for efficiently collecting, aggregating, and moving large amounts of log data from various sources to a centralized data store like HDFS. It uses a simple architecture based on streams of data flows. Each Flume data flow is built using three components: Source, Channel, and Sink.

- **Source:** Receives data from external sources (e.g., web servers).
- **Channel:** Acts as a temporary storage between Source and Sink.
- **Sink:** Delivers the data to the final destination (e.g., HDFS).

Prerequisites:

1. Apache Flume installed and configured on HDP Sandbox.
2. HDFS and YARN services are running.
3. Hive service (optional, if log analysis is required).
4. Access to HDP Sandbox through Putty or SSH terminal.

Detailed Steps to Perform the Practical

Step 1: Start HDP Sandbox and Connect via Putty

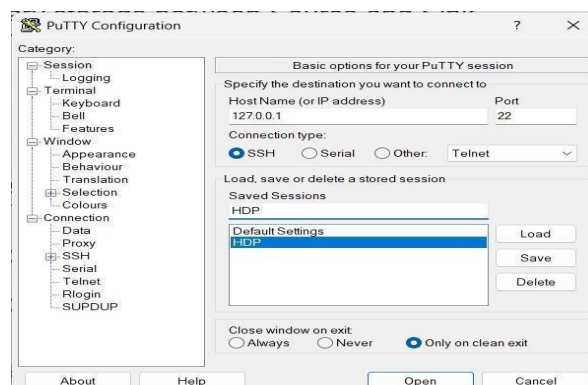
Open Putty and connect using the following credentials:

Hostname: sandboxhdp.hortonworks.com

Port: 22

Username: maria_dev

Password: maria_dev



Step 2: Create Directories in HDFS for Storing Logs

Run the following commands to create directories where Flume will store logs:

```
hdfs dfs -mkdir /flume_logs
```

```
hdfs dfs -mkdir /flume_logs/output
```

```
login as: maria_dev
maria_dev@127.0.0.1's password:
Last login: Sun Oct 26 06:13:56 2025 from 172.18.0.3
[maria_dev@sandbox-hdp ~]$ hdfs dfs -mkdir -p /flume_logs/output
[maria_dev@sandbox-hdp ~]$ hdfs dfs -ls /flume_logs
Found 1 items
drwxr-xr-x  - maria_dev hdfs          0 2025-10-26 06:28 /flume_logs/output
[maria_dev@sandbox-hdp ~]$
```

Step 3: Create a Sample Web Log File

Use the below commands to create and edit a sample log file: cd

```
/home/maria_dev
```

```
nano weblog.txt
```

Add the following log entries:

```
192.168.1.1 - - [06/Oct/2025:09:00:00 +0000] "GET /index.html HTTP/1.1"
200 1234
```

```
192.168.1.2 - - [06/Oct/2025:09:01:10 +0000] "POST /login HTTP/1.1" 302 567
```

```
192.168.1.3 - - [06/Oct/2025:09:02:33 +0000] "GET /products HTTP/1.1"
200 890
```

```
drwxr-xr-x  - maria_dev hdfs          0 2025-10-26 06:28 /flume_logs/output
[maria_dev@sandbox-hdp ~]$ cd /home/maria_dev
[maria_dev@sandbox-hdp ~]$ cat > weblog.txt <<'EOF'
> 192.168.1.1 - - [06/Oct/2025:09:00:00 +0000] "GET /index.html HTTP/1.1" 200 12
34
> 192.168.1.2 - - [06/Oct/2025:09:01:10 +0000] "POST /login HTTP/1.1" 302 567
> 192.168.1.3 - - [06/Oct/2025:09:02:33 +0000] "GET /products HTTP/1.1" 200 890
> EOF
[maria_dev@sandbox-hdp ~]$ cat weblog.txt
-bash: cat: command not found
[maria_dev@sandbox-hdp ~]$ cat weblog.txt
192.168.1.1 - - [06/Oct/2025:09:00:00 +0000] "GET /index.html HTTP/1.1" 200 1234
192.168.1.2 - - [06/Oct/2025:09:01:10 +0000] "POST /login HTTP/1.1" 302 567
192.168.1.3 - - [06/Oct/2025:09:02:33 +0000] "GET /products HTTP/1.1" 200 890
[maria_dev@sandbox-hdp ~]$
```

Step 4: Create a Flume Configuration File

Create a configuration file named **flume_web.conf** with the following content:

```
cd /home/maria_dev nano
```

flume_web.conf

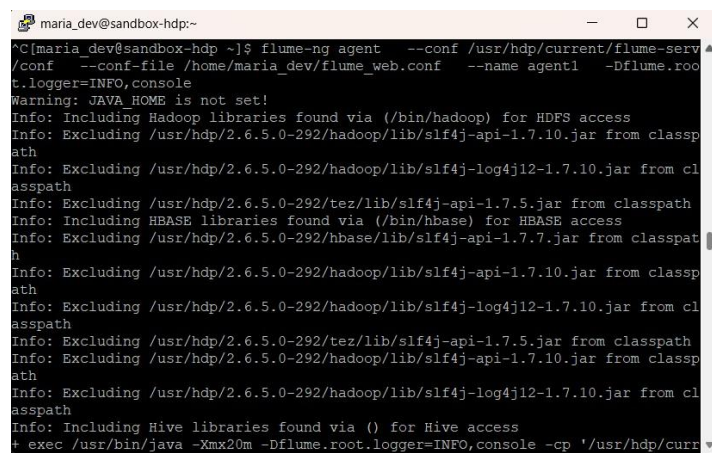
Add the following configuration:

```
# Define agent name agent1.sources = src1 agent1.sinks = sink1 agent1.channels =ch1 # Source
Configuration - monitor the log file agent1.sources.src1.type =
execagent1.sources.src1.command = tail -F
/home/maria_dev/weblog.txt # ChannelConfiguration agent1.channels.ch1.type = memory
agent1.channels.ch1.capacity =1000 agent1.channels.ch1.transactionCapacity = 100 # Sink
Configuration – write data to HDFS agent1.sinks.sink1.type = hdfs agent1.sinks.sink1.hdfs.path
hdfs://sandbox- hdp.hortonworks.com:8020/flume_logs/output agent1.sinks.sink1.hdfs.fileType
DataStreamagent1.sinks.sink1.hdfs.writeFormat= Text agent1.sinks.sink1.hdfs.batchSize = 100
agent1.sinks.sink1.hdfs.rollSize = 0 agent1.sinks.sink1.hdfs.rollCount = 10 # Bind Source and
Sink to Channel agent1.sources.src1.channels = ch1 agent1.sinks.sink1.channel = ch1
```

Step 5: Start the Flume Agent

Run the following command to start the Flume agent using your configuration file:

```
flume-ng agent \
--conf conf \
--conf-file /home/maria_dev/flume_web.conf \
--name agent1 \
```



```
maria_dev@sandbox-hdp:~$ flume-ng agent --conf /usr/hdp/current/flume-server/conf --conf-file /home/maria_dev/flume_web.conf --name agent1 -Dflume.root.logger=INFO,console
Warning: JAVA_HOME is not set!
Info: Including Hadoop libraries found via (/bin/hadoop) for HDFS access
Info: Excluding /usr/hdp/2.6.5.0-292/hadoop/lib/slf4j-api-1.7.10.jar from classpath
Info: Excluding /usr/hdp/2.6.5.0-292/hadoop/lib/slf4j-log4j12-1.7.10.jar from classpath
Info: Excluding /usr/hdp/2.6.5.0-292/tez/lib/slf4j-api-1.7.5.jar from classpath
Info: Including HBase libraries found via (/bin/hbase) for HBase access
Info: Excluding /usr/hdp/2.6.5.0-292/hbase/lib/slf4j-api-1.7.7.jar from classpath
Info: Excluding /usr/hdp/2.6.5.0-292/hadoop/lib/slf4j-api-1.7.10.jar from classpath
Info: Excluding /usr/hdp/2.6.5.0-292/hadoop/lib/slf4j-log4j12-1.7.10.jar from classpath
Info: Excluding /usr/hdp/2.6.5.0-292/tez/lib/slf4j-api-1.7.5.jar from classpath
Info: Excluding /usr/hdp/2.6.5.0-292/hadoop/lib/slf4j-api-1.7.10.jar from classpath
Info: Excluding /usr/hdp/2.6.5.0-292/hadoop/lib/slf4j-log4j12-1.7.10.jar from classpath
Info: Including Hive libraries found via () for Hive access
+ exec /usr/bin/java -Xmx20m -Dflume.root.logger=INFO,console -cp /usr/hdp/current/flume-server/lib/*: /usr/hdp/current/flume-server/bin/flume-ng-agent agent1
```

Step 6: Append New Logs to Simulate Streaming

In a new terminal window, add new log entries to simulate streaming log data:

```
echo '192.168.1.4 - - [06/Oct/2025:09:03:45 +0000] "GET /contact HTTP/1.1" 200
```

345' >> weblog.txt

Flume will automatically detect the new log entries and transfer them to HDFS.

```
login as: maria_dev
maria_dev@127.0.0.1's password:
Last login: Mon Oct 27 11:10:05 2025 from 172.18.0.3
[maria_dev@sandbox-hdp ~]$ echo '192.168.1.4 - - [06/Oct/2025:09:03:45 +0000] "GET /contact HTTP/1.1" 200 345' >> /home/maria_dev/weblog.txt
[maria_dev@sandbox-hdp ~]$ echo '192.168.1.5 - - [06/Oct/2025:09:04:10 +0000] "GET /index.html HTTP/1.1" 200 456' >> /home/maria_dev/weblog.txt
[maria_dev@sandbox-hdp ~]$
```

Step 7: Verify Logs in HDFS

Run the following commands to verify that the logs were successfully transferred to

HDFS: `hdfs dfs -ls /flume_logs/output` `hdfs dfs -cat /flume_logs/output/* | head -n 50`

```
[maria_dev@sandbox-hdp ~]$ hdfs dfs -ls /flume_logs/output
Found 3 items
-rw-r--r--  1 maria_dev hdfs      235 2025-10-27 11:19 /flume_logs/output/FlumeData.1761563955233
-rw-r--r--  1 maria_dev hdfs      157 2025-10-27 11:20 /flume_logs/output/FlumeData.1761564011703
-rw-r--r--  1 maria_dev hdfs      392 2025-10-27 11:21 /flume_logs/output/FlumeData.1761564068233
```

RESULT:-

Using Apache Flume, we can continuously collect and transfer log data from web servers into HDFS in real time. This data can then be processed and analyzed using Hive, Pig, or Spark to generate meaningful insights.

Flume simplifies log aggregation, ensuring reliability and scalability.

PRATICAL-09

Aim: Write a program to analyze the Spark File Management Tasks on Hadoop (HDP Sandbox).

Environment Setup Requirements:

1. Hortonworks HDP Sandbox (2.6 or above) installed and running on VirtualBox/VMware.
2. Putty or Terminal for SSH access.
3. Spark pre-installed in HDP under `/usr/hdp/current/spark2-client`.
4. Hadoop HDFS running inside sandbox. Step 1:

Start HDP Sandbox Services

1. Open browser @ <http://127.0.0.1:8080> @ login admin/admin
2. Start HDFS, YARN, Spark2 services from Ambari.
3. SSH into sandbox: `ssh root@127.0.0.1 -p 2222` (password: hadoop)

```
[maria_dev@sandbox-hdp ~]$ su root
Password:
[root@sandbox-hdp maria_dev]# cd/usr/hdp/current/spark2-client
```

Step 2: Create RDD using In-Memory Data

1. Go to Spark shell:
`cd /usr/hdp/current/spark2-client sparkshell`

```
[root@sandbox-hdp maria dev]# cd /usr/hdp/current/spark2-client
[root@sandbox-hdp spark2-client]# spark-shell
SPARK_MAJOR_VERSION is set to 2, using Spark2
Setting default log level to "WARN".
To adjust logging level use sc.setLogLevel(newLevel). For SparkR, use setLogLevel(newLevel).
Spark context Web UI available at http://sandbox-hdp.hortonworks.com:4040
Spark context available as 'sc' (master = local[*], app id = local-1761565354703).
Spark session available as 'spark'.
Welcome to

  ____  __
 / ___/  / /_  __
/ /   / __/ / / /
/ /___/ /_ / /_/ /
/_____/_/____/

 version 2.3.0.2.6.5.0-292

Using Scala version 2.11.8 (OpenJDK 64-Bit Server VM, Java 1.8.0_171)
Type in expressions to have them evaluated.
Type :help for more information.
```

2. Execute:

```
val data = Array(1,2,3,4,5) val rdd = sc.parallelize(data) rdd.collect()
```

```
scala> val data = Array(1,2,3,4,5)
data: Array[Int] = Array(1, 2, 3, 4, 5)

scala> val rdd = sc.parallelize(data)
rdd: org.apache.spark.rdd.RDD[Int] = ParallelCollectionRDD[0] at parallelize at
<console>:26

scala> rdd.collect()
res0: Array[Int] = Array(1, 2, 3, 4, 5)

scala>
```

Step 3: Create RDD using File

1. Create file in sandbox:

echo "Spark HDP Example" > sample.txt
hdfs dfs -mkdir /sparkdata
hdfs dfs -put sample.txt /sparkdata

```
[maria_dev@sandbox-hdp ~]$ echo "Spark HDP Example" > sample.txt
[maria_dev@sandbox-hdp ~]$ hdfs dfs -mkdir /sparkdata
[maria_dev@sandbox-hdp ~]$ hdfs dfs -put sample.txt /sparkdata
[maria_dev@sandbox-hdp ~]$ hdfs dfs -ls /sparkdata
Found 1 items
-rw-r--r-- 1 maria dev hdfs      18 2025-10-27 11:48 /sparkdata/sample.txt
[maria_dev@sandbox-hdp ~]$
```

2. Load file into RDD:

```
val fileRDD = sc.textFile("hdfs:///sparkdata/sample.txt") fileRDD.collect()
```

```
scala> val fileRDD = sc.textFile("hdfs:///sparkdata/sample.txt")
fileRDD: org.apache.spark.rdd.RDD[String] = hdfs:///sparkdata/sample.txt MapPart
itionsRDD[1] at textFile at <console>:24

scala> fileRDD.collect()
res0: Array[String] = Array(Spark HDP Example)
```

Step 4: Map Function Example

1. Use RDD map:

```
val numbers = sc.parallelize(1 to 5)
```

```
val squareRDD = numbers.map(x => x * x) squareRDD.collect()
```

```
scala> val numbers = sc.parallelize(1 to 5)
numbers: org.apache.spark.rdd.RDD[Int] = ParallelCollectionRDD[2] at parallelize
at <console>:24

scala> val squareRDD = numbers.map(x => x * x)
squareRDD: org.apache.spark.rdd.RDD[Int] = MapPartitionsRDD[3] at map at <consol
e>:25

scala> squareRDD.collect()
res1: Array[Int] = Array(1, 4, 9, 16, 25)
```

Step 5: Reduce Function Example

1. Use reduce function: val sum = numbers.reduce((a, b) => a + b) println(sum)

```
scala> val sum = numbers.reduce((a, b) => a + b)
sum: Int = 15

scala> println(sum)
15
```

RESULT:-

This practical covered: 4 Creating RDD from in-memory data, 4 Creating RDD from HDFS file, 4 Using map() transformation, 4 Using reduce() action, 4 Saving output to HDFS.

PRATICAL-10

AIM: To perform the HBase File Management Practical on HDP Sandbox.

Detailed Step-by-Step Procedure with Explanations

Objective

Perform basic HBase NoSQL operations using HDP Sandbox: create table, insert records, and display table data.

Prerequisites

HDP Sandbox installed and running in VirtualBox/VMware. Ambari and HBase services started.

Step 1: Start HDP Sandbox

Power on the HDP Sandbox. Open Ambari at <http://localhost:8080>

(login:maria_dev/maria_dev) and ensure HBase, HDFS, and ZooKeeper are running.

Step 2: Connect to Sandbox via SSH

Open terminal and run: `ssh root@127.0.0.1 -p 2222` (password: hadoop).

Step 3: Start HBase Shell

Run command: `hbase shell`. This opens HBase CLI for NoSQL operations.

```
[maria_dev@sandbox-hdp ~]$ su root
Password:
[root@sandbox-hdp maria_dev]# hbase shell
HBase Shell; enter 'help<RETURN>' for list of supported commands.
Type "exit<RETURN>" to leave the HBase Shell
Version 1.1.2.2.6.5.0-292, r897822d4dd5956ca186974c10382e9094683fa29, Fri May 11
 08:00:59 UTC 2018

hbase(main):001:0> █
```

Step 4: Create Table

Command: `create 'student', 'personal', 'academic'`. Creates table with two column families.

```
hbase(main):001:0> create 'student', 'personal', 'academic'
0 row(s) in 6.5180 seconds

=> Hbase::Table - student
hbase(main):002:0> list
TABLE
ATLAS_ENTITY_AUDIT_EVENTS
atlas_titan
student
3 row(s) in 0.7680 seconds

=> ["ATLAS_ENTITY_AUDIT_EVENTS", "atlas_titan", "student"]
hbase(main):003:0> █
```

Step 5: Insert Records

Commands:


```
put 'student','1','personal:name','Alice'
```

```
put 'student','1','academic:marks','85'
```

```
put 'student','2','personal:name','Bob'
```

```
put 'student','2','academic:marks','90'
```

```
=> ["ATLAS ENTITY AUDIT EVENTS", "atlas_titan", "student"]
hbase(main):003:0> put 'student','1','personal:name','Alice'
0 row(s) in 6.9520 seconds

hbase(main):004:0> put 'student','1','academic:marks','85'
0 row(s) in 0.1000 seconds

hbase(main):005:0> put 'student','2','personal:name','Bob'
0 row(s) in 0.0470 seconds

hbase(main):006:0> put 'student','2','academic:marks','90'
0 row(s) in 0.2370 seconds
```

Step 6: Display Table Data

Scan full table: scan 'student'.

```
hbase(main):007:0> scan 'student'
ROW          COLUMN+CELL
 1           column=academic:marks, timestamp=1761566918570, value=85
 1           column=personal:name, timestamp=1761566917320, value=Alice
 2           column=academic:marks, timestamp=1761566919729, value=90
 2           column=personal:name, timestamp=1761566919221, value=Bob
2 row(s) in 0.5810 seconds
```

View specific row: get 'student','1'.

```
hbase(main):010:0> get 'student','1'
COLUMN      CELL
academic:marks    timestamp=1761566918570, value=85
personal:name     timestamp=1761566917320, value=Alice
2 row(s) in 2.0320 seconds
```

Step 7: List Tables

Command: list. Displays all tables present in HBase.

```
hbase(main):011:0> list
TABLE
ATLAS_ENTITY_AUDIT_EVENTS
atlas_titan
student
3 row(s) in 0.0340 seconds

=> ["ATLAS ENTITY AUDIT EVENTS", "atlas_titan", "student"]
hbase(main):012:0>
```

Conclusion

Successfully executed HBase NoSQL operations on HDP Sandbox including creating table, inserting data, and viewing records.